Linked Lists

The Presentation

Ruhan Alpaydin

Faculty of Computer and Information Sciences
Yeditepe University

v2026-11-07

Outline

1. Why a linked list?
2. Nodes and Edges
3. What You Pay and What You Gain
4. Doubly Linked
5. Uses and Wrap-up

Why a linked list?

Two ways to store a sequence

Array

`a[0]`, `a[1]`, . . . , `a[n-1]` in one block.
Jump to `a[k]` in $O(1)$.
Insert or delete in the middle: move $n - k$ elements.

Linked list

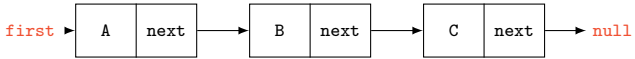
Each element lives in its own **node**, with a `next` reference.
Jump to position k: walk from the head ($O(k)$).
Insert or delete: change two references.

Neither is better: an array is better when you know the position; a linked list is better when you know the node.

What a node is

Field	Meaning	Java
data	the value being stored	`int iData`
next	a reference to the next node, or `null`	`Link next`

A linked list is a chain of such nodes. The list holds a single reference, `first`, to the head of the chain. The last node in the chain has `next == null`.



This list has three nodes: A, B, C, then `null`.

Nodes and Edges

ADT with nodes and edges

Structure	Shape	Node has
Linked list	linear	`next`
Tree	hierarchical	`left`, `right` (or `children`)
Heap	complete binary tree with the heap order	`left`, `right`
Graph	non-linear	a list of neighbours

All four are the same idea: a set of nodes, each holding data and a set of edges to other nodes. The linked list is the simplest one.

A tree is a linked list with two nexts

- Each node has `left` and `right`.
- `null` means “no child there”.
- Traversal is a walk, not a jump: you still go through references.
- Binary search tree: order by key.

What changes, what stays

Same node idea.
Same reference-by-reference traversal.
Different order: by depth, by key, by BFS/DFS.

A heap is a tree with an order

- Complete binary tree: filled left to right, level by level.
- Heap order: parent $\leq$ child (min-heap) or parent $\geq$ child (max-heap).
- Same node and reference idea; different invariant.
- The `PriorityQueue` in the Java API is a binary heap stored in an array, not in nodes.

The invariant is the data structure

A linked list has no invariant between neighbours. A heap has one. A search tree has one. That is what makes them different algorithms on the same node-and-reference idea.

A graph is nodes and edges, unrestricted

- Each node has a list of adjacent nodes.
- No linear order is implied.
- Directed or undirected; a self-loop is an edge from a node to itself.
- BFS and DFS run on graphs and use — respectively — a queue and a stack.

The linked list, the tree and the heap are special cases of a graph. The stack and the queue can also be drawn as graphs; the next slides show why.

First run: a chain and its traversal

codeoutput/NodeChain.java — Java

```java
public class NodeChain {

    static class Node {
        final int item;
        final Node next;

        Node(int item, Node next) {
            this.item = item;
        this.next = next;
        }
    }

    public static void main(String[] args) {

        Node third = new Node(30, null);
        Node second = new Node(20, third);
        Node first  = new Node(10, second);

        for (Node p = first; p != null; p = p.next)
            System.out.println(p.item);
    }
}
```

Terminal — codeoutput/NodeChain.txt

```
$ javac NodeChain.java
$ java NodeChain
10
20
30
```

Forward only: there is no `chain.get(3)`.

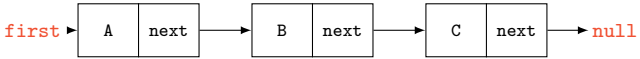
What You Pay and What You Gain

The trade

Operation	Array	Linked list
access `a[k]`	$O(1)$	$O(k)$
insert at front	$O(n)$	$O(1)$
delete at front	$O(n)$	$O(1)$
insert at rear	$O(1)$ amort.	$O(1)$ with `last`
insert in middle (given the node)	$O(n)$	$O(1)$
search by value	$O(n)$	$O(n)$
memory	contiguous, compact	one node per element

The trick: insert and delete change two references, not the elements. The rest of the list never moves.

Structure of a singly linked list



- One reference (`first`) is enough to find every node.
- The list has no `last` unless you add one.
- Every node is reached through the previous node's `next`.

You can only walk forward

Sequential access

To reach position k, start at `first` and follow `next` k times. There is no `a[k]` in a linked list.

- Iteration is the normal way to visit every element.
- Any algorithm that jumps by index is the wrong fit.
- The JDK's `LinkedList.get(i)` is $O(i)$, which is why you use it as a `Deque`, not as a `List`.

Insert at the head

1. Create the new node.
2. Point its `next` to the old `first`.
3. Point `first` at the new node.

Two references

The rest of the list is untouched. $O(1)$.

Delete at the head

1. Save the old `first`.
2. Advance `first` to `first.next`.
3. Return the saved node's data.

One reference

The removed node becomes unreachable; the GC reclaims it. $O(1)$.

Insert at the tail

- Without a `last` reference: walk from `first` to the end, then attach. $O(n)$.
- With a `last` reference: attach at `last`, then move `last`. $O(1)$.

Double-ended list

`FirstLastList` keeps both `first` and `last`. That is the whole difference between a stack (only needs `first`) and a queue (needs both).

The JDK gives you both: `ArrayList` and `LinkedList`

The abstract array and the abstract linked list have concrete names in `java.util`.

`ArrayList<E>`: the array

- one resizable `Object[]`
- `get(i)`: $O(1)$
- `add(0, x)`: $O(n)$ (shift right)
- `remove(0)`: $O(n)$ (shift left)
- iterate: cache-friendly

`LinkedList<E>`: the chain

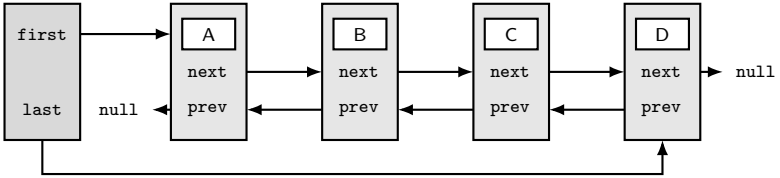
- one node per element, `next` and `prev`
- `get(i)`: $O(i)$ (walk from `first`)
- `add(0, x)`: $O(1)$ (relink)
- `remove(0)`: $O(1)$ (relink)
- iterate: pointer chasing

Same `List<E>` interface, different internals

`List<Integer> xs = new ArrayList<>();` and `List<Integer> xs = new LinkedList<>();` are interchangeable at the call site. The choice is about *workload*, not about the API. `LinkedList` is doubly linked, which is why it can also be used as a `Deque` — see the next section.

Doubly Linked

A node with two references



- Each node holds three fields: `data`, `next`, `prev`.
- The list keeps both ends: `first` at the head, `last` at the tail.
- Cost: one extra reference per element.
- Benefit: every operation on the two ends is $O(1)$, and iteration can go either way.

What prev buys you

codeoutput/JavaContainers.java — Java

```java
import java.util.LinkedList;

public class JavaContainers {
    public static void main(String[] args) {
        LinkedList<String> list = new LinkedList<>();
        list.addFirst("b");
        list.addFirst("a");
        list.addLast("c");

        System.out.println("list:       " + list);
        System.out.println("getFirst(): " + list.getFirst());
        System.out.println("getLast():  " + list.getLast());

        // Peel from the tail. Each removeLast() is O(1) because of prev;
        // a singly linked list would have to walk from head every time.
        System.out.print("peel:       ");
        while (!list.isEmpty()) {
            System.out.print(list.removeLast() + " ");
        }
        System.out.println();
        System.out.println("list:       " + list);
    }
}
```

Terminal — codeoutput/JavaContainers.txt

```
$ javac JavaContainers.java
$ java JavaContainers
list:       [a, b, c]
getFirst(): a
getLast():  c
peel:       c b a
list:       []
```

- `getLast()` and `removeLast()` would need a walk on a singly linked list; here they go through `last.prev` and are $O(1)$.
- Peeling from the back (`removeLast()`) is symmetric to peeling from the front.

Uses and Wrap-up

Where a linked list wins

- FIFO or LIFO where the number of elements is not known in advance.
- Splicing two sequences (LRU cache, undo/redo).
- Music playlists (insert in the middle, skip to next).
- The bucket chain of a hash table.
- Adjacency lists of a graph.

Where it loses

- Random access by index.
- Cache-sensitive hot loops.
- Anything where you know the array size up front.

Big ideas

- A linked list is a chain of **nodes**, each with a data field and a `next` reference.
- The list object holds only `first`; `last` is optional and makes enqueue $O(1)$.
- A stack is a linked list with only `first`; a queue is a linked list with `first` and `last`.
- A doubly linked list adds `prev`, so deletion no longer needs the previous node.
- Trees, heaps and graphs are the same node-and-edge idea with an invariant. A linked list is the simplest case.
- You cannot jump; every access walks.

Check yourself

1. What does a node hold that an array element does not?
2. Why is `insertFirst` $O(1)$ but `get(k)` $O(k)$?
3. Why does a queue need `last` but a stack does not?
4. What does a doubly linked node hold that a singly linked node does not?
5. Name three of the four node-and-edge structures from the PPTX.
6. When would you pick `LinkedList` over `ArrayList` in the JDK?

References

1. Robert Lafore, *Data Structures & Algorithms in Java*, 2nd ed., Sams, 2002.
   The `Link` / `LinkList` / `FirstLastList` style reused in the freshman deck.
2. Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein, *Introduction to Algorithms*, 4th ed., MIT Press, 2022.
   Chapter 10: elementary data structures, linked lists, stacks, queues.
3. Robert Sedgewick and Kevin Wayne, *Algorithms*, 4th ed., Addison-Wesley, 2011.
   Section 1.3: linked lists; Section 3.1: symbol tables built on them.
4. *8 Basic Data Structures Every Programmer Should Know*, GoDaddy Resources.
   `https://www.godaddy.com/resources/in/web-pro-in/8-basic-data-structures-every-programmer-should-know`
   The source for the node-and-edge survey (list, tree, heap, graph) on slides 3–7 of the deck.
5. Oracle, *Java SE 21 API Specification*.
   `java.util.LinkedList`, `java.util.List`, `java.util.Deque`.
6. Oracle, *The Java Tutorials: Trail: Collections*.

The Two Exercises from previous years

Stutter: duplicate every element

missing file: codeoutput/Stutter.java

missing file: codeoutput/Stutter.txt

- Build a new list with each element twice, then swap it in.
- `[1, 2, 3, 4]` $\longrightarrow$ `[1, 1, 2, 2, 3, 3, 4, 4]`.
- One pass, $O(n)$ time, $O(n)$ extra space.

Mirror: append the reverse of itself

```
missing file:  codeoutput/Mirror.java
```

```
missing file:  codeoutput/Mirror.txt
```

- Copy the list, reverse the copy, append it.
- `[1, 2, 3, 4]` $\longrightarrow$ `[1, 2, 3, 4, 4, 3, 2, 1]`.
- Symmetric input is a palindrome; the deque version needs no copy.

Classic Problems

Reverse in place

```
missing file:  codeoutput/ReverseList.java
```

```
missing file:
codeoutput/ReverseList.txt
```

- Three pointers, one pass: `prev`, `cur`, `next`.
- Redirect each `next` backwards, then advance.
- $O(n)$ time, $O(1)$ space, no new nodes.

Palindrome check

```
missing file:  codeoutput/Palindrome.java
```

```
missing file:
codeoutput/Palindrome.txt
```

- Two indices, one from each end, meet in the middle.
- `r a c e c a r` $\longrightarrow$ `true`; `a b c` $\longrightarrow$ `false`.
- On a linked list without `get(i)`, the same idea needs a stack or a recursion.

Josephus problem

missing file: codeoutput/Josephus.java

missing file: codeoutput/Josephus.txt

- n people in a circle, every k-th is removed.
- For $n = 7$, $k = 3$, the survivor is 4.
- A circular list is the natural model; the array version is $O(n^2)$, the linked version $O(nk)$.

Questions?